

Практикум: нормализация данных

Модель «снежинка» в PostgreSQL

Цель: спроектировать схему «снежинка» и реализовать загрузку денормализованных данных из CSV в нормализованную реляционную модель со справочными таблицами (dim_country, dim_city, dim_category).

2026 г.

1 Постановка задачи

1.1 Исходные данные

Вам предоставлен набор из 10 CSV-файлов (директория исходные данные/), каждый содержит 1000 строк с полностью денормализованными данными о продажах зоотоваров. Все файлы содержат одинаковый набор из 50 столбцов.

Структура полей CSV:

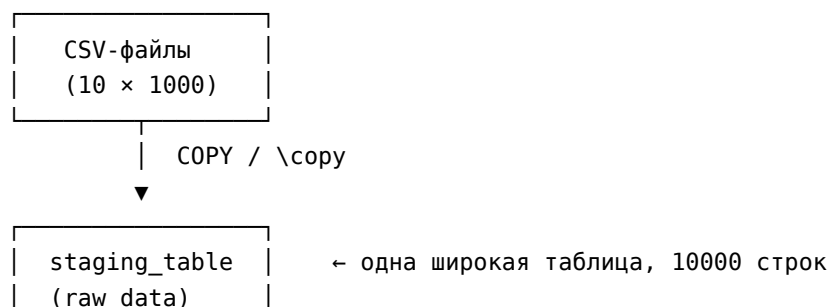
Группа полей	Поля
Идентификатор	id (1-1000)
Данные клиента	customer_first_name, customer_last_name, customer_age, customer_email, customer_country, customer_postal_code, customer_pet_type, customer_pet_name, customer_pet_breed
Данные продавца	seller_first_name, seller_last_name, seller_email, seller_country, seller_postal_code
Данные товара	product_name, product_category, product_price, product_quantity, pet_category, product_weight, product_color, product_size, product_brand, product_material, product_description, product_rating, product_reviews, product_release_date, product_expiry_date
Данные продажи	sale_date, sale_customer_id, sale_seller_id, sale_product_id, sale_quantity, sale_total_price
Данные магазина	store_name, store_location, store_city, store_state, store_country, store_phone, store_email
Данные поставщика	supplier_name, supplier_contact, supplier_email, supplier_phone, supplier_address, supplier_city, supplier_country

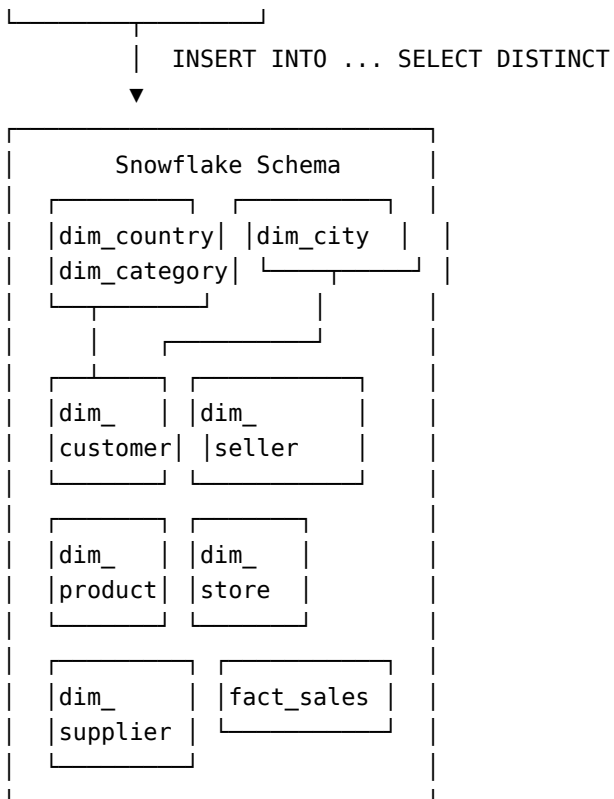
Каждая строка — полностью денормализованная запись: данные о клиенте, продавце, товаре, магазине, поставщике и самой продаже объединены в одну строку.

Вопрос для самопроверки: Что означает «денормализованная» запись? Какие данные в строке дублируются между разными строками? Обратите внимание: страна повторяется для клиентов, продавцов, магазинов и поставщиков — как эту избыточность устраняет «снежинка»?

1.2 Что нужно построить

Реляционную схему «снежинка» в PostgreSQL и скрипты для загрузки данных:





1.3 Требования к результату

1. docker-compose.yml — PostgreSQL + автоматический импорт CSV в staging-таблицу.
2. DDL-скрипт (sql/03_ddl.sql) — CREATE TABLE для staging и всех 8 таблиц схемы «снежинка» (3 справочника + 5 измерений + факт).
3. DML-скрипт (sql/04_dml.sql) — INSERT INTO ... SELECT для заполнения таблиц из staging в правильном порядке зависимостей.
4. В итоговой схеме должно быть 10000 уникальных фактов продаж.
5. Аналитические запросы должны работать корректно (JOIN фактов с измерениями и справочниками).

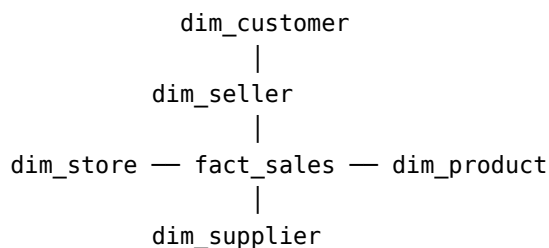
2 Краткая теория

2.1 Схема «звезда» (Star Schema)

Паттерн проектирования хранилищ данных. Данные разделяются на два типа таблиц:

- **Таблицы измерений** (dimension) — описательные атрибуты: кто, что, где. Пример: dim_customer с полями имени, возраста, страны (текстовое поле country).
- **Таблица фактов** (fact) — измеримые события (продажи) с числовыми мерами (сумма, количество) и внешними ключами на измерения.

Структура связей «звезды»:



В схеме «звезда» каждая таблица измерений **денормализована** — все атрибуты хранятся в одной таблице. Например, в dim_customer поле country хранится как обычный текст.

2.2 Схема «снежинка» (Snowflake Schema) — наш выбор

Вариант «звезды», в котором таблицы измерений **нормализуются** дальше. Повторяющиеся текстовые значения выносятся в отдельные справочные таблицы с суррогатными ключами. В нашем проекте:

```
dim_country ← dim_customer
dim_country ← dim_seller
dim_category ← dim_product
dim_country ← dim_city ← dim_store
dim_country ← dim_city ← dim_supplier
```

fact_sales

Что изменилось по сравнению со «звездой»:

Таблица	«Звезда» (было)	«Снежинка» (стало)
dim_customer	country TEXT	country_id INTEGER FK → dim_country
dim_seller	country TEXT	country_id INTEGER FK → dim_country
dim_product	category TEXT	category_id INTEGER FK → dim_category
dim_store	city, state, country TEXT	city_id INTEGER FK → dim_city
dim_supplier	city, country TEXT	city_id INTEGER FK → dim_city

Добавлены 3 справочные таблицы:

Справочник	Первичный ключ	Кто ссылается
dim_country	country_id SERIAL	dim_customer, dim_seller, dim_city
dim_city	city_id SERIAL	dim_store, dim_supplier

dim_category	category_id SERIAL	dim_product
--------------	--------------------	-------------

Вопрос для самопроверки: В чём преимущество «снежинки» перед «звездой»? (Подсказка: что если название страны изменится — сколько строк нужно обновить в каждом варианте?) А в чём недостаток «снежинки» с точки зрения аналитических запросов?

2.3 Суррогатные ключи

Суррогатный ключ — искусственный первичный ключ (обычно SERIAL или IDENTITY), не имеющий бизнес-смысла. Он используется вместо естественных ключей из исходных данных.

Преимущества:

- Не зависит от изменений в бизнес-данных.
- Целочисленный ключ эффективнее для JOIN и индексов.
- Устойчив к дублированию и изменению исходных ID.

В «снежинке» суррогатные ключи играют **ключевую роль**: справочные таблицы (dim_country, dim_city, dim_category) получают SERIAL-ключи, а измерения ссылаются на них через INTEGER FK.

Подсказка: Для клиента, продавца и товара в CSV есть поля id, sale_customer_id, sale_seller_id, sale_product_id — их можно использовать как ключи. Но для магазина, поставщика, страны, города и категории уникального ID в CSV нет — dim_country, dim_city, dim_category используют SERIAL, а dim_store/dim_supplier генерируют ID через хеш-функцию hashtext().

2.4 Нормализация в «снежинке» на примере проекта

Рассмотрим конкретные примеры нормализации в нашем проекте:

Пример 1: Страна. В CSV поля customer_country, seller_country, store_country, supplier_country содержат одни и те же значения (например, «USA»). В «звезде» каждая таблица хранит текст страны — избыточность. В «снежинке» создаётся dim_country с country_id, и все таблицы ссылаются на него. Если «USA» нужно переименовать в «United States of America» — меняется одна строка.

Пример 2: Город. Поля store_city, store_state, store_country и supplier_city, supplier_country содержат географические данные. В «снежинке» dim_city хранит (name, state, country_id) с уникальным ограничением UNIQUE (name, state, country_id), а dim_store и dim_supplier ссылаются на city_id.

Пример 3: Категория товара. Поле product_category содержит повторяющиеся значения («Food», «Toys» и т.д.). В «снежинке» dim_category хранит уникальные категории, а dim_product содержит category_id FK → dim_category.

Вопрос для самопроверки: Сколько уровней нормализации разумно применить? Почему в нашем проекте `dim_city` ссылается на `dim_country`, а не содержит текст страны напрямую? Когда «снежинка» становится слишком сложной?

3 Этап 1. Docker + PostgreSQL

Задание: Создайте `docker-compose.yml` с сервисом PostgreSQL. Настройте автоматический импорт CSV-файлов в `staging`-таблицу.

3.1 Требования к `docker-compose.yml`

1. Образ: `postgres:16`.
2. Задайте переменные окружения: `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`.
3. Пробросьте порт 5432 на хост.
4. Примонтируйте директорию `./sql/` в `/docker-entrypoint-initdb.d/` — скрипты выполняются автоматически при первом запуске контейнера.
5. Добавьте `healthcheck` с `pg_isready`.

Подсказка:

```
postgres:
  image: postgres:16
  environment:
    POSTGRES_DB: petstore
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
  ports:
    - "5432:5432"
  volumes:
    - ./sql:/docker-entrypoint-initdb.d
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 5
```

3.2 Staging-таблица

Создайте таблицу `staging_sales` (или `raw_data`) с колонками, соответствующими всем 50 полям CSV. Тип `TEXT` для всех полей — это допустимо для `staging`; точные типы будут у таблиц схемы «снежинка».

Подсказка:

```
CREATE TABLE IF NOT EXISTS raw_data (
  file_id    INTEGER,
  id         TEXT,
  customer_first_name TEXT,
  customer_last_name TEXT,
  -- ... все 50 полей ...
);
```

Поле `file_id` нужно для различения строк из разных файлов (ID пересекаются).

3.3 Импорт CSV-файлов

Скрипты в `/docker-entrypoint-initdb.d/` выполняются в алфавитном порядке. Вы можете:

1. Создать 00_create_staging.sql — DDL для staging-таблицы.
2. Создать 01_import.sh — shell-скрипт, который копирует CSV в контейнер и вызывает COPY.
3. 03_ddl.sql — DDL таблиц «снежинки».
4. 04_dml.sql — DML для заполнения таблиц.

Подсказка: Для импорта из shell-скрипта используйте команду `psql -c "\copy raw_data FROM '/path/to/file.csv' WITH (FORMAT csv, HEADER true)"`. Обратите внимание: `\copy` (с обратным слешем) — клиентская команда, не требующая прав супер-пользователя.

Вопрос для самопроверки: Почему `COPY` (серверная команда) может не работать для импорта файлов с хоста? Чем отличается `\copy`?

4 Этап 2. Импорт данных

Задание: Загрузите все 10 CSV-файлов (10000 строк) в единую staging-таблицу.

4.1 Проблема пересекающихся ID

Внимание: Все 10 CSV-файлов содержат одинаковый диапазон id от 1 до 1000. Если импортировать их без модификации, первичный ключ будет конфликтовать или вы получите дубликаты.

Решения:

1. Добавить колонку `file_id` (номер файла 0–9) и использовать составной ключ (`file_id, id`).
2. Добавить колонку `row_id` SERIAL — автоинкрементный ID для каждой строки.
3. При импорте добавлять смещение: файл с индексом `i` получает `offset = i * 1000`.

Подсказка: Если используете `\copy` в shell-скрипте, можно задать `file_id` через `psql -c "UPDATE raw_data SET file_id = $i WHERE file_id IS NULL"` после каждого импорта или добавить колонку со значением по умолчанию перед `\copy`. В данном проекте используется `raw_data` как имя staging-таблицы.

4.2 Проверка импорта

Контрольная точка: После импорта выполните:

```
SELECT COUNT(*) FROM raw_data;
```

Ожидаемый результат: 10000 строк.

```
SELECT file_id, COUNT(*) FROM raw_data GROUP BY file_id ORDER BY file_id;
```

Ожидаемый результат: 10 файлов по 1000 строк каждый.

5 Этап 3. Анализ данных

Задание: Исследуйте staging-таблицу, чтобы понять структуру данных перед проектированием схемы «снежинка».

5.1 Разведочные запросы

Выполните следующие запросы и запишите результаты:

```
-- Сколько всего строк?
SELECT COUNT(*) FROM raw_data;

-- Сколько уникальных клиентов?
SELECT COUNT(DISTINCT customer_email) FROM raw_data;

-- Сколько уникальных продавцов?
SELECT COUNT(DISTINCT seller_email) FROM raw_data;

-- Сколько уникальных товаров?
SELECT COUNT(DISTINCT product_name) FROM raw_data;

-- Сколько уникальных магазинов?
SELECT COUNT(DISTINCT store_name || store_city) FROM raw_data;

-- Сколько уникальных поставщиков?
SELECT COUNT(DISTINCT supplier_name || supplier_city) FROM raw_data;

-- Есть ли NULL или пустые строки в ключевых полях?
SELECT
    COUNT(*) FILTER (WHERE customer_email IS NULL OR customer_email = '')
    AS null_emails,
    COUNT(*) FILTER (WHERE sale_total_price IS NULL OR sale_total_price = '')
    AS null_prices;
```

5.2 Анализ для справочных таблиц «снежинки»

Дополнительно исследуйте данные, которые пойдут в справочники:

```
-- Сколько уникальных стран (из всех источников)?
SELECT COUNT(*) FROM (
    SELECT DISTINCT customer_country FROM raw_data
    WHERE customer_country IS NOT NULL AND customer_country != ''
    UNION
    SELECT DISTINCT seller_country FROM raw_data
    WHERE seller_country IS NOT NULL AND seller_country != ''
    UNION
    SELECT DISTINCT store_country FROM raw_data
    WHERE store_country IS NOT NULL AND store_country != ''
    UNION
    SELECT DISTINCT supplier_country FROM raw_data
    WHERE supplier_country IS NOT NULL AND supplier_country != ''
) countries;

-- Сколько уникальных категорий товаров?
SELECT COUNT(DISTINCT product_category) FROM raw_data
WHERE product_category IS NOT NULL AND product_category != '';
```

```
-- Сколько уникальных городов (магазины + поставщики)?
SELECT COUNT(*) FROM (
    SELECT DISTINCT store_city FROM raw_data
    WHERE store_city IS NOT NULL AND store_city != ''
    UNION
    SELECT DISTINCT supplier_city FROM raw_data
    WHERE supplier_city IS NOT NULL AND supplier_city != ''
) cities;
```

Подсказка: Эти запросы покажут, сколько строк будет в dim_country, dim_category и dim_city соответственно. В «снежинке» справочники небольшие (десятки-сотни строк), но на них ссылаются тысячи записей в измерениях — именно это даёт экономию при обновлениях.

5.3 Поиск дубликатов

```
-- Дубликаты магазинов
SELECT store_name, store_city, store_country, COUNT(*)
FROM raw_data
GROUP BY store_name, store_city, store_country
HAVING COUNT(*) > 1
ORDER BY COUNT(*) DESC
LIMIT 10;

-- Дубликаты поставщиков
SELECT supplier_name, supplier_city, supplier_country, COUNT(*)
FROM raw_data
GROUP BY supplier_name, supplier_city, supplier_country
HAVING COUNT(*) > 1
ORDER BY COUNT(*) DESC
LIMIT 10;
```

Вопрос для самопроверки: Один и тот же магазин (по названию и городу) встречается в нескольких строках. В скольких? Что это значит для проектирования таблицы dim_store? А как город и страна будут вынесены в dim_city и dim_country?

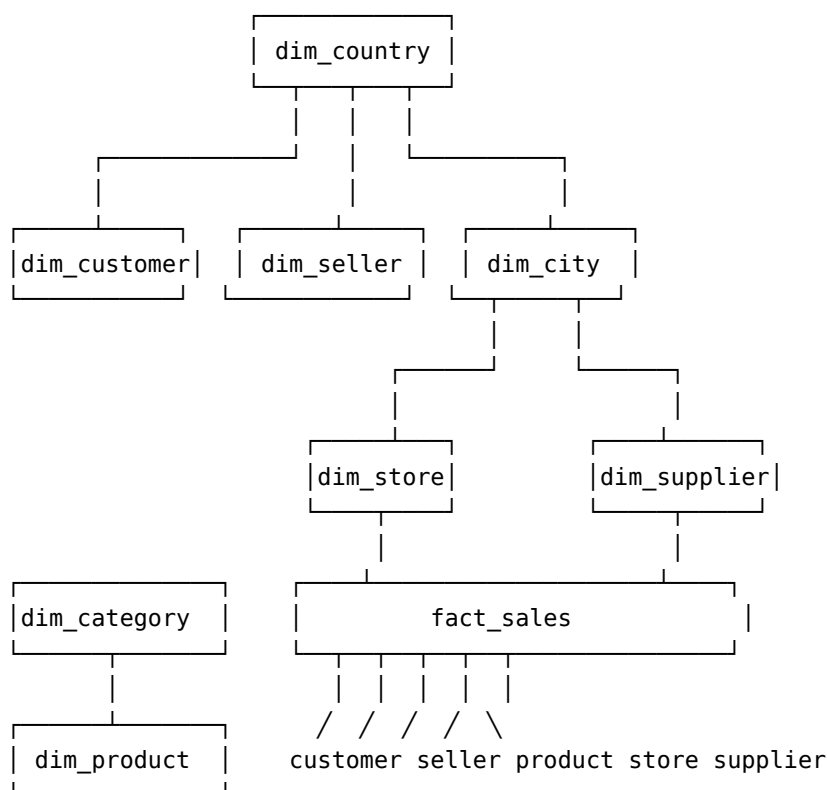
6 Этап 4. Проектирование схемы «снежинка»

Задание: Определите справочные таблицы, таблицы измерений и таблицу фактов. Нарисуйте схему «снежинки» с зависимостями.

6.1 Полный список таблиц (8 + staging + факт)

Таблица	Тип	Описание
dim_country	Справочник	Уникальные страны: country_id SERIAL, name VARCHAR(100) UNIQUE
dim_city	Справочник	Города: city_id SERIAL, name, state, country_id FK → dim_country
dim_category	Справочник	Категории: category_id SERIAL, name VARCHAR(100) UNIQUE
dim_customer	Измерение	Клиенты: ключ id из CSV, country_id FK → dim_country
dim_seller	Измерение	Продавцы: ключ id из CSV, country_id FK → dim_country
dim_product	Измерение	Товары: ключ id из CSV, category_id FK → dim_category
dim_store	Измерение	Магазины: суррогатный ключ (хеш), city_id FK → dim_city
dim_supplier	Измерение	Поставщики: суррогатный ключ (хеш), city_id FK → dim_city
fact_sales	Факт	Продажи: FK на все 5 измерений

6.2 Диаграмма связей



6.3 Таблица фактов

fact_sales — содержит:

1. Внешние ключи: customer_id, seller_id, product_id, store_id, supplier_id.

2. Меры: quantity, total_price.
3. Атрибут: sale_date.

6.4 Суррогатные ключи

В «снежинке» используется три вида ключей:

1. SERIAL (автоинкремент) — для dim_country, dim_city, dim_category. PostgreSQL назначает ID автоматически при INSERT.
2. Целые числа из CSV — для dim_customer (id), dim_seller (id), dim_product (id).
- Хеш-функция hashtext() — для dim_store и dim_supplier (детерминированный ID по комбинации полей).

Подсказка: Для dim_store хеш генерируется так:

```
(abs(hashtext(
  COALESCE(store_name, '') || '|' ||
  COALESCE(store_location, '') || '|' ||
  COALESCE(store_city, '') || '|' ||
  COALESCE(store_state, '') || '|' ||
  COALESCE(store_country, '') || '|' ||
  COALESCE(store_phone, '') || '|' ||
  COALESCE(store_email, '')
)) % 2147483647) AS store_id
```

hashtext() даёт детерминированный integer — одинаковые входные данные всегда дают одинаковый ID. % 2147483647 гарантирует положительное значение.

Вопрос для самопроверки: Почему для dim_store и dim_supplier выбран хеш, а не SERIAL? (Подсказка: как потом связать строки в fact_sales с правильным store_id?)

7 Этап 5. DDL-скрипты

Задание: Напишите SQL-скрипт `sql/03_ddl.sql` для создания всех таблиц «снежинки».

7.1 Порядок создания (DDL)

Внимание: Таблицы нужно создавать в порядке зависимостей: сначала таблицы без FK, потом ссылающиеся на них, и так далее. При удалении — обратный порядок.

Порядок CREATE TABLE:

1. `dim_country` — нет FK.
2. `dim_city` — FK → `dim_country`.
3. `dim_category` — нет FK.
4. `dim_customer` — FK → `dim_country`.
5. `dim_seller` — FK → `dim_country`.
6. `dim_product` — FK → `dim_category`.
7. `dim_store` — FK → `dim_city`.
8. `dim_supplier` — FK → `dim_city`.
9. `fact_sales` — FK → все 5 измерений (последняя!).

Порядок DROP TABLE (обратный):

1. `fact_sales`
2. `dim_supplier`, `dim_store`, `dim_product`, `dim_seller`, `dim_customer`
3. `dim_category`, `dim_city`, `dim_country`

Подсказка: Шаблон DDL для справочной таблицы:

```
CREATE TABLE dim_country (  
    country_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) UNIQUE NOT NULL  
);
```

Для dim_city — составной уникальный ключ:

```
CREATE TABLE dim_city (  
    city_id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    state VARCHAR(100),  
    country_id INTEGER REFERENCES dim_country(country_id),  
    UNIQUE (name, state, country_id)  
);
```

Для измерения с FK на справочник:

```
CREATE TABLE dim_customer (  
    customer_id INTEGER PRIMARY KEY,  
    first_name VARCHAR(100),  
    last_name VARCHAR(100),  
    age INTEGER,  
    email VARCHAR(200),  
    country_id INTEGER REFERENCES dim_country(country_id),  
    postal_code VARCHAR(20),  
    pet_type VARCHAR(50),  
    pet_name VARCHAR(100),  
    pet_breed VARCHAR(100)  
);
```

Подсказка: Шаблон DDL для таблицы фактов:

```
CREATE TABLE fact_sales (  
    sale_id INTEGER PRIMARY KEY,  
    customer_id INTEGER REFERENCES dim_customer(customer_id),  
    seller_id INTEGER REFERENCES dim_seller(seller_id),  
    product_id INTEGER REFERENCES dim_product(product_id),  
    store_id INTEGER REFERENCES dim_store(store_id),  
    supplier_id INTEGER REFERENCES dim_supplier(supplier_id),  
    sale_date DATE,  
    quantity INTEGER,  
    total_price DECIMAL(10,2)  
);
```

Вопрос для самопроверки: Почему таблицу фактов нужно создавать последней? Что будет, если создать её до таблиц измерений? А что будет, если dim_city создать до dim_country?

8 Этап 6. DML-скрипты

Задание: Напишите SQL-скрипт `sql/04_dml.sql` для заполнения всех таблиц «снежинки» из staging в правильном порядке.

8.1 Ключевой принцип: порядок вставки зависит от FK

Внимание: Таблицы с внешними ключами можно заполнять **только после** того, как заполнены таблицы, на которые они ссылаются. Нарушение порядка → ошибка `violates foreign key constraint`.

Порядок INSERT:

1. `dim_country` ← INSERT без JOIN (чистый DISTINCT из `raw_data`)
2. `dim_city` ← INSERT с JOIN `dim_country` (нужен `country_id`)
3. `dim_category` ← INSERT без JOIN (чистый DISTINCT из `raw_data`)
4. `dim_customer` ← JOIN `dim_country` (нужен `country_id`)
5. `dim_seller` ← JOIN `dim_country` (нужен `country_id`)
6. `dim_product` ← JOIN `dim_category` (нужен `category_id`)
7. `dim_store` ← JOIN `dim_city` (нужен `city_id`)
8. `dim_supplier` ← JOIN `dim_city` (нужен `city_id`)
9. `fact_sales` ← JOIN всех 5 измерений

8.2 Заполнение `dim_country`

Собираем уникальные страны из всех четырёх источников:

```
INSERT INTO dim_country (name)
SELECT DISTINCT customer_country
FROM raw_data
WHERE customer_country IS NOT NULL AND customer_country != ''
UNION
SELECT DISTINCT seller_country
FROM raw_data
WHERE seller_country IS NOT NULL AND seller_country != ''
UNION
SELECT DISTINCT store_country
FROM raw_data
WHERE store_country IS NOT NULL AND store_country != ''
UNION
SELECT DISTINCT supplier_country
FROM raw_data
WHERE supplier_country IS NOT NULL AND supplier_country != '';
```

Подсказка: Обратите внимание: UNION (без ALL) автоматически убирает дубликаты между разными источниками. Страна «USA» из `customer_country` и «USA» из `store_country` станет одной записью в `dim_country`.

8.3 Заполнение `dim_city`

Нужен JOIN с `dim_country`, чтобы получить `country_id`:

```
INSERT INTO dim_city (name, state, country_id)
SELECT DISTINCT
    COALESCE(NULLIF(store_city, ''), '(unknown)'),
```



```

        COALESCE(NULLIF(store_state, ''), '(unknown)'),
        co.country_id
FROM raw_data r
JOIN dim_country co
    ON COALESCE(NULLIF(r.store_country, ''), '(unknown)') = co.name
WHERE r.store_city IS NOT NULL AND r.store_city != '';
-- ... аналогично для supplier_city ...

```

Подсказка: dim_city должна быть заполнена **после** dim_country, иначе JOIN dim_country не найдёт country_id и city_id будет NULL. Обратите внимание на обработку пустых значений: COALESCE(NULLIF(..., ''), '(unknown)') заменяет пустые строки на '(unknown)', чтобы JOIN работал корректно.

8.4 Заполнение dim_category

Простейший INSERT — просто уникальные значения из одного поля:

```

INSERT INTO dim_category (name)
SELECT DISTINCT product_category
FROM raw_data
WHERE product_category IS NOT NULL AND product_category != '';

```

8.5 Заполнение dim_customer, dim_seller, dim_product

Эти таблицы требуют JOIN со справочниками для получения FK:

```

-- dim_customer: JOIN dim_country для country_id
INSERT INTO dim_customer (customer_id, first_name, last_name, age, email,
    country_id, postal_code, pet_type, pet_name, pet_breed)
SELECT DISTINCT ON (r.id)
    r.id, ...
    co.country_id,
    ...
FROM raw_data r
LEFT JOIN dim_country co ON r.customer_country = co.name
ORDER BY r.id;

```

Подсказка: LEFT JOIN (а не INNER JOIN) используется потому, что в raw_data могут быть записи с пустой или отсутствующей страной. При LEFT JOIN такие записи получают country_id = NULL, что допустимо. При INNER JOIN они были бы потеряны.

8.6 Заполнение dim_store и dim_supplier

Суррогатный ключ генерируется через hashtext():

```

INSERT INTO dim_store (store_id, name, location, city_id, phone, email)
SELECT
    (abs(hashtext(
        COALESCE(store_name, '') || '|' ||
        COALESCE(store_location, '') || '|' ||
        COALESCE(store_city, '') || '|' ||
        COALESCE(store_state, '') || '|' ||
        COALESCE(store_country, '') || '|' ||
        COALESCE(store_phone, '') || '|' ||
        COALESCE(store_email, '')
    )) % 2147483647) AS store_id,

```

```

    NULLIF(store_name, ''),
    NULLIF(store_location, ''),
    dc.city_id,
    NULLIF(store_phone, ''),
    NULLIF(store_email, '')
FROM (SELECT DISTINCT ... FROM raw_data) r
LEFT JOIN dim_city dc ON ...

```

Подсказка: dim_store и dim_supplier требуют JOIN dim_city, а dim_city ссылается на dim_country. Поэтому порядок: dim_country → dim_city → dim_store/dim_supplier. Каждый уровень «снежинки» добавляет ещё один JOIN в DML.

8.7 Заполнение fact_sales

Самый сложный этап — нужно связать каждую строку staging с ID во всех измерениях:

```

INSERT INTO fact_sales (sale_id, customer_id, seller_id, product_id,
    store_id, supplier_id, sale_date, quantity, total_price)
SELECT
    r.id,
    r.sale_customer_id,
    r.sale_seller_id,
    r.sale_product_id,
    ds.store_id,
    dsp.supplier_id,
    TO_DATE(NULLIF(r.sale_date, ''), 'MM/DD/YYYY'),
    r.sale_quantity,
    r.sale_total_price
FROM raw_data r
JOIN dim_store ds ON ...      -- по комбинации полей магазина
JOIN dim_supplier dsp ON ...  -- по комбинации полей поставщика

```

Внимание: Для dim_store и dim_supplier JOIN производится по комбинации текстовых полей (название, город, контакт и т.д.), а не по ID. Для dim_customer, dim_seller, dim_product — по числовому id из CSV. Если в измерениях есть дубликаты, JOIN вернёт несколько строк на одну продажу — проверьте уникальность!

Вопрос для самопроверки: Почему для dim_store и dim_supplier нельзя использовать простой JOIN по id? Чем это отличается от dim_customer?

8.8 Обработка типов данных

Данные в staging хранятся как TEXT. При вставке нужно привести типы:

Подсказка:

```
-- Безопасное приведение даты (формат MM/DD/YYYY в CSV)
TO_DATE(NULLIF(s.sale_date, ''), 'MM/DD/YYYY')

-- Безопасное приведение числа (с обработкой пустых строк)
NULLIF(s.product_price, '')::DECIMAL(10,2)

-- NULLIF превращает пустую строку в NULL, что позволяет избежать ошибки CAST
```

Контрольная точка: После выполнения DML-скрипта проверьте:

```
SELECT 'fact_sales' AS t, COUNT(*) FROM fact_sales UNION ALL
SELECT 'dim_customer', COUNT(*) FROM dim_customer UNION ALL
SELECT 'dim_seller', COUNT(*) FROM dim_seller UNION ALL
SELECT 'dim_product', COUNT(*) FROM dim_product UNION ALL
SELECT 'dim_store', COUNT(*) FROM dim_store UNION ALL
SELECT 'dim_supplier', COUNT(*) FROM dim_supplier UNION ALL
SELECT 'dim_country', COUNT(*) FROM dim_country UNION ALL
SELECT 'dim_city', COUNT(*) FROM dim_city UNION ALL
SELECT 'dim_category', COUNT(*) FROM dim_category;
```

fact_sales должно содержать 10000 строк. Справочники (dim_country, dim_city, dim_category) — небольшое число строк.

9 Этап 7. Проверка

Задание: Выполните аналитические запросы и убедитесь, что данные корректны.

9.1 Контроль целостности

```
-- Все ли факты имеют ссылки на измерения?
SELECT COUNT(*) AS broken FROM fact_sales f
LEFT JOIN dim_customer c ON f.customer_id = c.customer_id
WHERE c.customer_id IS NULL;

-- Все ли страны из измерений есть в dim_country?
SELECT COUNT(*) AS orphan_countries
FROM dim_customer c
LEFT JOIN dim_country co ON c.country_id = co.country_id
WHERE co.country_id IS NULL;

-- Все ли города из dim_store есть в dim_city?
SELECT COUNT(*) AS orphan_cities
FROM dim_store s
LEFT JOIN dim_city dc ON s.city_id = dc.city_id
WHERE dc.city_id IS NULL;

-- Сумма выручки
SELECT SUM(total_price) FROM fact_sales;

-- Количество уникальных дат
SELECT COUNT(DISTINCT sale_date) FROM fact_sales;
```

Контрольная точка: broken = 0, orphan_countries = 0, orphan_cities = 0, сумма выручки — осмысленное число, даты > 1.

9.2 Аналитические запросы (с учётом «снежинки»)

В «снежинке» для доступа к атрибутам из справочников требуется **многотабличный JOIN**. Например, чтобы получить страну клиента, нужно пройти: fact_sales → dim_customer → dim_country.

1. **Топ-10 стран по выручке** — fact_sales JOIN dim_customer ON ... JOIN dim_country ON dim_customer.country_id = dim_country.country_id, GROUP BY dim_country.name, SUM(total_price).
2. **Продажи по категориям товаров** — fact_sales JOIN dim_product ON ... JOIN dim_category ON dim_product.category_id = dim_category.category_id, GROUP BY dim_category.name.
3. **Продажи по городам магазинов** — fact_sales JOIN dim_store ON ... JOIN dim_city ON dim_store.city_id = dim_city.city_id, GROUP BY dim_city.name.
4. **Количество продаж по месяцам** — DATE_TRUNC('month', sale_date), GROUP BY, ORDER BY.
5. **Топ-5 продавцов по объёму продаж** — fact_sales JOIN dim_seller, SUM(total_price).

Подсказка: Пример запроса «Топ стран по выручке» — обратите внимание на два JOIN:

```
SELECT co.name AS country,
       ROUND(SUM(f.total_price), 2) AS revenue,
       COUNT(*) AS sales_count
FROM fact_sales f
JOIN dim_customer c ON f.customer_id = c.customer_id
JOIN dim_country co ON c.country_id = co.country_id
GROUP BY co.name
ORDER BY revenue DESC
LIMIT 10;
```

В «звезде» хватило бы одного JOIN (прямое поле country в dim_customer). В «снежинке» нужен дополнительный JOIN через справочник.

Подсказка: Пример запроса «Продажи по городам магазинов» — три JOIN:

```
SELECT ci.name AS city,
       ci.state,
       co.name AS country,
       COUNT(*) AS sales,
       ROUND(SUM(f.total_price), 2) AS revenue
FROM fact_sales f
JOIN dim_store s ON f.store_id = s.store_id
JOIN dim_city ci ON s.city_id = ci.city_id
JOIN dim_country co ON ci.country_id = co.country_id
GROUP BY ci.name, ci.state, co.name
ORDER BY revenue DESC
LIMIT 10;
```

Вопрос для самопроверки: Напишите запрос, который показывает продажи по месяцам с разбивкой по категориям товаров. Сколько JOIN потребуется? Сравните с аналогичным запросом для «звезды».

10 Типичные проблемы и решения

Симптом	Причина / решение
ERROR: duplicate key value	Попытка вставить строку с уже существующим первичным ключом. Проверьте, что используете DISTINCT или смещение ID для разных файлов.
ERROR: insert or update on table violates foreign key constraint	Ссылка на несуществующую запись. Проверьте порядок INSERT: сначала справочники (dim_country, dim_city, dim_category), потом измерения, потом факты. Убедитесь, что dim_city заполняется после dim_country.
ERROR: invalid input syntax for type numeric	Пустая строка вместо числа. Используйте NULLIF(field, '')::DECIMAL.
ERROR: invalid input syntax for type date	Некорректный формат даты. В CSV даты в формате ММ/ДД/YYYY — используйте TO_DATE(field, 'ММ/ДД/YYYY').
В fact_sales 20000 строк вместо 10000	JOIN с dim_store или dim_supplier вернул дубликаты (несколько записей с одинаковыми полями). Проверьте уникальность измерений: SELECT col1, col2, COUNT(*) FROM dim_store GROUP BY col1, col2 HAVING COUNT(*) > 1.
COPY не видит CSV-файлы	Серверная команда COPY ищет файлы внутри контейнера. Используйте \copy (клиентская) или примонтируйте CSV в нужную директорию.
dim_city содержит NULL в country_id	dim_country не была заполнена до dim_city. Проверьте порядок INSERT в DML-скрипте.
fact_sales содержит NULL в store_id или supplier_id	JOIN с dim_store/dim_supplier не нашёл соответствие. Проверьте, что в справочных таблицах есть все нужные записи. Используйте LEFT JOIN для диагностики.
После перезапуска контейнера данные пропали	Скрипты в /docker-entrypoint-initdb.d/ выполняются только при первой инициализации. Если нужно пересоздать — удалите том: docker-compose down -v.

11 Контрольные вопросы

11.1 Теория: Star vs Snowflake

1. Объясните разницу между схемой «звезда» и «снежинка» на примере данного проекта (как `country` хранится в «звезде» vs «снежинке»?).
2. Какие преимущества даёт «звезда»? А какие — «снежинка»? Когда стоит выбрать «снежинку»?
3. Что такое таблица фактов? Какие данные в ней хранятся?
4. Что такое таблица измерений? Чем она отличается от справочной таблицы в «снежинке»?
5. Что такое суррогатный ключ? Зачем он нужен? В каких таблицах нашего проекта он используется?

11.2 Нормализация

1. Какая нормальная форма соответствует «звезде»? А «снежинке»?
2. В нашем проекте `dim_country` вынесена отдельно, а `pet_type` — нет. Почему? Как определить, какие поля стоит нормализовать?
3. Почему `dim_city` ссылается на `dim_country`, а не хранит текст страны напрямую? Как это влияет на аналитические запросы?
4. Что такое денормализация? Зачем она применяется в хранилищах данных? Когда «снежинка» хуже «звезды»?
5. В `dim_city` используется `UNIQUE (name, state, country_id)`. Почему не просто `UNIQUE (name)`? Может ли быть два города с одинаковым названием?

11.3 DDL и проектирование

1. В каком порядке нужно создавать таблицы «снежинки» с внешними ключами? Почему `dim_city` создаётся после `dim_country`?
2. В каком порядке нужно удалять таблицы? Что делает `CASCADE`?
3. Объясните назначение `REFERENCES`, `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`.
4. Почему тип `SERIAL` удобен для справочных таблиц (`dim_country`, `dim_city`, `dim_category`)?
5. Как обеспечить уникальность данных в таблице измерений (кроме `PRIMARY KEY`)?

11.4 DML и загрузка данных

1. В каком порядке нужно выполнять `INSERT` для «снежинки»? Что будет, если вставить данные в `dim_customer` до заполнения `dim_country`?
2. Чем отличается `INSERT INTO ... SELECT` от обычного `INSERT INTO ... VALUES`?
3. Зачем нужен `DISTINCT` при заполнении таблиц измерений и справочников?
4. Как работает `LEFT JOIN` при заполнении `dim_customer`? Почему не `INNER JOIN`?
5. Как привести тип `TEXT` к `DATE`, `DECIMAL`, `INTEGER` в PostgreSQL? Что делает `NULLIF`?
6. Объясните, как `hashtext()` генерирует детерминированный ключ для `dim_store`. Почему важен модуль % 2147483647?

11.5 Аналитические запросы в «снежинке»

1. Напишите запрос «Топ-5 стран по выручке» для «снежинки». Сколько JOIN потребуется?
2. Напишите запрос «Продажи по категориям товаров» с учётом `dim_category`.
3. Чем отличается аналитический запрос в «снежинке» от аналогичного в «звезде»?
4. Как найти факты, у которых нет соответствующей записи в справочнике (`dim_country`)?
5. Что делает `DATE_TRUNC('month', sale_date)`? Как её использовать для отчёта по месяцам?

11.6 Практические вопросы

1. Зачем нужен staging-слой? Почему не загружать данные сразу в «снежинку»?
2. Как автоматизировать импорт CSV через `/docker-entrypoint-initdb.d/`?
3. Что произойдёт при повторном запуске `docker-compose up`? Почему данные не дублируются?
4. Как проверить корректность данных после загрузки? Назовите минимум 3 проверки для «снежинки».

12 Справочник

12.1 Docker Compose

Команда	Описание
<code>docker-compose up -d --build</code>	Запустить все сервисы в фоне
<code>docker-compose down -v</code>	Остановить + удалить тома (сброс БД)
<code>docker ps</code>	Запущенные контейнеры
<code>docker logs <container></code>	Логи контейнера
<code>docker exec -it <id> psql -U postgres -d petstore</code>	Подключиться к БД
<code>docker volume ls</code>	Список томов

12.2 Полезные SQL-паттерны

-- Количество записей во всех таблицах «снежинки»

```
SELECT 'raw_data' AS t, COUNT(*) FROM raw_data UNION ALL
SELECT 'fact_sales', COUNT(*) FROM fact_sales UNION ALL
SELECT 'dim_customer', COUNT(*) FROM dim_customer UNION ALL
SELECT 'dim_seller', COUNT(*) FROM dim_seller UNION ALL
SELECT 'dim_product', COUNT(*) FROM dim_product UNION ALL
SELECT 'dim_store', COUNT(*) FROM dim_store UNION ALL
SELECT 'dim_supplier', COUNT(*) FROM dim_supplier UNION ALL
SELECT 'dim_country', COUNT(*) FROM dim_country UNION ALL
SELECT 'dim_city', COUNT(*) FROM dim_city UNION ALL
SELECT 'dim_category', COUNT(*) FROM dim_category;
```

-- Безопасное приведение типов

```
NULLIF(text_field, '')::INTEGER
NULLIF(text_field, '')::DECIMAL(10,2)
TO_DATE(NULLIF(date_text, ''), 'MM/DD/YYYY')
```

-- Генерация детерминированного ID через хеш

```
(abs(hashtext(col1 || '|' || col2 || '|' || col3)) % 2147483647)
```

-- Проверка дубликатов

```
SELECT col1, col2, COUNT(*)
FROM table_name
GROUP BY col1, col2
HAVING COUNT(*) > 1;
```

-- Upsert (insert or ignore)

```
INSERT INTO table (id, name) VALUES (1, 'test')
ON CONFLICT (id) DO NOTHING;
```

-- Проверка целостности FK в «снежинке»

```
SELECT COUNT(*) AS broken
FROM dim_customer c
LEFT JOIN dim_country co ON c.country_id = co.country_id
WHERE co.country_id IS NULL;
```

12.3 Аналитические запросы для «снежинки» (шаблоны)

-- Топ стран по выручке (два JOIN: fact → customer → country)

```
SELECT co.name AS country,  
       ROUND(SUM(f.total_price), 2) AS revenue  
FROM fact_sales f  
JOIN dim_customer c ON f.customer_id = c.customer_id  
JOIN dim_country co ON c.country_id = co.country_id  
GROUP BY co.name  
ORDER BY revenue DESC  
LIMIT 10;
```

-- Продажи по категориям (два JOIN: fact → product → category)

```
SELECT cat.name AS category,  
       COUNT(*) AS cnt,  
       ROUND(AVG(f.total_price), 2) AS avg_price  
FROM fact_sales f  
JOIN dim_product p ON f.product_id = p.product_id  
JOIN dim_category cat ON p.category_id = cat.category_id  
GROUP BY cat.name  
ORDER BY cnt DESC;
```

-- Продажи по городам магазинов (три JOIN: fact → store → city → country)

```
SELECT ci.name AS city,  
       co.name AS country,  
       ROUND(SUM(f.total_price), 2) AS revenue  
FROM fact_sales f  
JOIN dim_store s ON f.store_id = s.store_id  
JOIN dim_city ci ON s.city_id = ci.city_id  
JOIN dim_country co ON ci.country_id = co.country_id  
GROUP BY ci.name, co.name  
ORDER BY revenue DESC  
LIMIT 10;
```

-- Динамика по месяцам

```
SELECT DATE_TRUNC('month', sale_date) AS month,  
       COUNT(*) AS sales,  
       ROUND(SUM(total_price), 2) AS revenue  
FROM fact_sales  
GROUP BY month  
ORDER BY month;
```

-- Полная сводка: продажи по стране клиента, категории, городу магазина

```
SELECT co.name AS customer_country,  
       cat.name AS product_category,  
       ci.name AS store_city,  
       COUNT(*) AS sales,  
       ROUND(SUM(f.total_price), 2) AS revenue  
FROM fact_sales f  
JOIN dim_customer c ON f.customer_id = c.customer_id  
JOIN dim_country co ON c.country_id = co.country_id  
JOIN dim_product p ON f.product_id = p.product_id  
JOIN dim_category cat ON p.category_id = cat.category_id  
JOIN dim_store s ON f.store_id = s.store_id  
JOIN dim_city ci ON s.city_id = ci.city_id
```

```
GROUP BY co.name, cat.name, ci.name  
ORDER BY revenue DESC  
LIMIT 20;
```